Beginning with the expression following the `switch` statement, we will read the state of the digital input pin 2. If the tilt switch is activated then the pin will read as `HIGH`, which if you remember is basically equal to the value 1, and the code for case 1 will be executed. If the tilt switch is not activated it will read as `LOW` or 0, so the code beginning with case 0 will be executed until the end of the `switch` statement. This will also include the code following `case 1` because the normal program flow did not reach the closing curly brace yet.

What this means is that, assuming the `switch` statement is the only code in our `loop()` function, when the tilt switch is off, the LED will blink on and off every 1 second but when the tilt switch is on, the LED will stay on and not turn off again until the tilt switch is turned off.

Break

The `break` statement can be used inside other control statements to immediately end the loop or statement. If we borrow our five-blink sample from before, we could add an `if` statement followed by the `break` statement to exit out of the loop if a switch is activated. This would look like the following:

```
for (int i = 0; i < 5; i++) {
  if (digitalRead(2) == HIGH) break;
  digitalWrite(13, HIGH);
  delay(250);
  digitalWrite(13, LOW);
  delay(250);
}
```

Normally, in this example the `for` loop will complete its cycle five times before continuing on with the rest of the program. The additional line `if (digitalRead(2) == HIGH) break;` will check the status of pin 2 and if it is equal to `HIGH`, from a switch being activated, the break statement will cause the `for` loop to quit immediately and program control will resume with the next line of code after the `for` loop.

To go back to the earlier example for the switch statement, if rather than the LED blinking when the switch is `LOW`, we could use the break statement to break out of the switch statement by adding the `break` keyword at the end of case 0, like in the following modified example:

```
switch (digitalRead(2)) {
  case 0:
    digitalWrite(13, LOW);
    delay(1000);
    break;
  case 1:
    digitalWrite(13, HIGH);
    delay(1000);
}
```

Using `break`, when the code for case 0 has been executed, the statement is broken and program flow continues with the code after the end of the `switch` statement. With the modified code, we should be left with an LED that is on when the switch is on and off when the switch is off. The added bonus with the `break` statement is that it gives us the ability to have a loop that is conditional on multiple factors or behaviors. This is also effectively a way to end any loop structure through external sources.